NICOLE MOOLMAN

DOE522 FA 2

QUESTION 1: RELEASE PIPELINE DESIGN (15 MARKS)

Stages:

Build Stage- Compile the code, execute unit tests, and generate build artifacts.
Activated by commits to specified branches or pull requests.
Tools include Azure Pipelines, Jenkins, and any other continuous integration tool.

Testing Stage-Run integration, UI, and performance tests.
A staging environment can be used to test the new version.
Canary deployments and feature toggles are two examples of

Deployment strategies:

Canary deployments- Start with a small selection of users and progressively increase the percentage as confidence grows.
Feature Toggles: New features are released in a dormant condition and enabled progressively.

Deployment Stage:

Blue-Green Deployment involves maintaining two similar environments (blue and green), routing traffic to the blue environment and deploying to the green. Once approved, switch traffic to green.
If problems emerge, traffic can be routed back to the blue environment for a rollback.

Continuous monitoring:

Keep track of application performance and issues as they occur during deployment.
In the post-deployment stage, run tests and monitor the application for any difficulties. User acceptance testing (UAT) and beta testing are also options.
Automation tools include Azure Monitor and Application Insights.

Triggers:

Include code commits, merge requests, and specified planned times.
Manual approvals may be necessary at specific phases, such as before production deployment.

Deployment Strategy:

Canary Deployment is ideal for testing with a small number of users.
Blue-Green Deployment: Minimizes downtime by keeping two identical environments.

Feature Toggles:

Allows for the selective delivery of features without full deployments.

QUESTION 2: ROLLBACK STRATEGY (15 MARKS).

Steps for Rollback:

1. **Immediate Identification:**
    - Set up monitoring and alerting to detect issues quickly.
    - Tools: Azure Monitor, Application Insights.
2. **Deploy Previous Stable Version:**
    - Use version control to identify the last stable release.
    - Redeploy this version using the CI/CD pipeline.
    - For **Blue-Green Deployments**, switch traffic back to the previous environment.
3. **Database Rollback:**
    - If database changes were made, have a backup or migration script ready to revert changes.
    - Tools: Entity Framework migrations, SQL scripts.
4. **Automated Rollback:**
    - Use automation tools to revert deployments quickly and reduce manual intervention.
    - Tools: Azure Pipelines, Terraform.
5. **Communication:**
    - Notify stakeholders about the rollback and its implications.
    - Maintain clear documentation for the rollback process.

Tools and Techniques:

- **Version Control Systems** (Git): To revert to previous versions of code.
- **Deployment Automation** (Azure Pipelines, Jenkins): To automate and speed up the rollback process.
- **Database Backup/Restore**: To ensure the database state aligns with the codebase.

QUESTION 3: SECURITY MEASURES IN PIPELINES (20 MARKS)

Secure Access Control:
Use Azure Active Directory (Azure AD) for role-based access control (RBAC).
Limit pipeline and resource access to just approved users.

Secret Management:
In Azure Key Vault, you may store sensitive information including API keys, connection strings, and passwords.
Azure Pipelines provides secure access to secrets via service connections.

Pipeline security:
Implement branch policies to enforce code review and approval procedures.
Use signed commits and code signing to ensure code integrity.

Audit Logging:

Allow logging of pipeline activity to track changes and detect unwanted access or adjustments.
Tools include Azure Monitor and Azure Security Centre.

Compliance with Security Best Practices:
Use HTTPS for all communications.
Patch vulnerabilities by regularly updating dependencies and libraries.
Use static code analysis to identify vulnerabilities early on in the development process.

Question 4: Compliance and Auditing (15 marks).

Enforcing policies:
Use Azure Policy to ensure compliance throughout your Azure environment.
Define and enforce policies governing resource configuration, tagging, and data residency.

Audit logs:
Enable auditing on Azure resources to record and store activity logs.
Use Azure Monitor and Azure Security Center to continuously monitor and alert on compliance issues.
Compliance Standards:

Integrate checks into your CI/CD workflow to ensure compliance with industry standards like as GDPR, HIPAA, and ISO.
Azure Blueprints, for example, can help you manage compliance by specifying a repeatable set of Azure resources.
Change Management:

Implement change management methods to monitor and approve changes in the pipeline.
Tools include ServiceNow integration with Azure Pipelines for change management.

Question 5: Infrastructure as Code Benefits (10 marks)

Benefits of IaC:

Consistency and repeatability
Infrastructure is defined in code, so environments remain consistent between deployments.
For example, use Terraform to deploy identical environments across development, staging, and production.

Automation and Speed:
IaC allows you to automate infrastructure provisioning and configuration, which reduces setup time.
For example, in Azure, you can use ARM templates to automatically provision an entire environment in minutes.

Version Control and Collaboration:
Infrastructure specifications can be versioned, just like application code, allowing teams to collaborate and track changes.
For example, to manage infrastructure modifications, Terraform scripts might be stored on GitHub.

Question 6: DSC Configuration Management (25 marks)

1. Define DSC Configurations

To start, you'll need to define the desired configuration for your VMs using a DSC configuration script written in PowerShell. This script specifies the desired state for various aspects of your VMs, such as installed software, services, and file settings.

```
configuration WebServer
{
    param (
        [string[]]$NodeName = 'localhost',
        [string]$InstallPath = 'C:\inetpub\wwwroot'
    )

    Node $NodeName
    {
        # Install the IIS role
        WindowsFeature IIS
        {
            Ensure = 'Present'
            Name = 'Web-Server'
        }

        # Ensure the website directory exists
        File WebsiteFolder
        {
            Ensure = 'Present'
            Type = 'Directory'
            DestinationPath = $InstallPath
        }

        # Ensure the default webpage is present
        File DefaultPage
        {
            Ensure = 'Present'
            DestinationPath = "$InstallPath\index.html"
            Contents = 'Welcome to Azure DSC Managed Web Server'
        }

        # Ensure the IIS service is running
        Service IISService
        {
            Name = 'W3SVC'
            Ensure = 'Running'
            StartUpType = 'Automatic'
            DependsOn = '[WindowsFeature]IIS'
```

2. Compile the Configuration

Once the configuration script is defined, you need to compile it into a .mof file, which is the format used by DSC.

```
# Compile the configuration into a .mof file
WebServer -OutputPath "C:\DSC\WebServer"
```

This command will generate a .mof file for each node (VM) that the configuration is applied to.

3. Apply the Configuration

To apply the configuration to the VMs, you can use Azure Automation or manually push the configuration using PowerShell.

Using Azure Automation:

1. **Create an Automation Account**: In the Azure portal, create an Automation Account.
2. **Import the DSC Configuration**: Upload your configuration script to the Azure Automation Account.
3. **Compile the Configuration**: Compile the DSC configuration within Azure Automation, which generates the .mof files.
4. **Assign the Configuration to VMs**: Link the compiled configuration to your VMs in the multi-node environment.

```
# Apply the configuration to the node
Start-DscConfiguration -Path "C:\DSC\WebServer" -Wait -Verbose -Force
```

This command pushes the compiled configuration to the target VM and applies it.

4. Handle Configuration Drift

Configuration drift occurs when the state of the VM deviates from the desired configuration. DSC automatically checks the configuration of the nodes periodically and corrects any drift. However, you can manually initiate a consistency check.

Monitor and Correct Drift:

- **Azure Automation**: Azure Automation can be configured to report on configuration drift and automatically correct it.
- **PowerShell**: You can manually check and correct drift by reapplying the configuration.

```
# Check for drift and apply correction if needed
Test-DscConfiguration -Detailed
Start-DscConfiguration -UseExisting -Wait -Verbose
```

5. Automation and Reporting

To streamline the process, you can automate these steps using Azure Automation Runbooks. Additionally, you can configure alerts and reports to monitor the state of your configurations.

- **Automation Runbooks**: Automate the deployment, monitoring, and drift correction processes using Runbooks in Azure Automation.
- **Reporting**: Use Azure Monitor or other tools to generate reports on the state of your VMs, including any configuration drift detected and corrected.